Week 3 - Univariate Analysis

Please run the cells of the notebook as you get to them while reading

In [2]:
```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
```

1. Lesson: Bar charts and univariate graphs

Let's make a dataset (in this case, just a series) which is weighted to have more small values than large values. By squaring a random number between 0 and 1, we ensure that half (those whose initial value is below 0.5) are below 0.25, while the other half are between 0.25 and 1. This means that most of the values are small, and it's more interesting than analyzing a perfectly uniform dataset. This kind of trick - transforming one random variable to get another - can generate a variety of random datasets for you. We then multiply by 100 to get a number between 0 and 100.

In [3]:
```
np.random.seed(0)
lesson_series = np.round(np.random.random(size = 1000)**2 * 100, 2)
lesson_series[0:10] # check the first ten values.  Are they mostly on the small sid
```

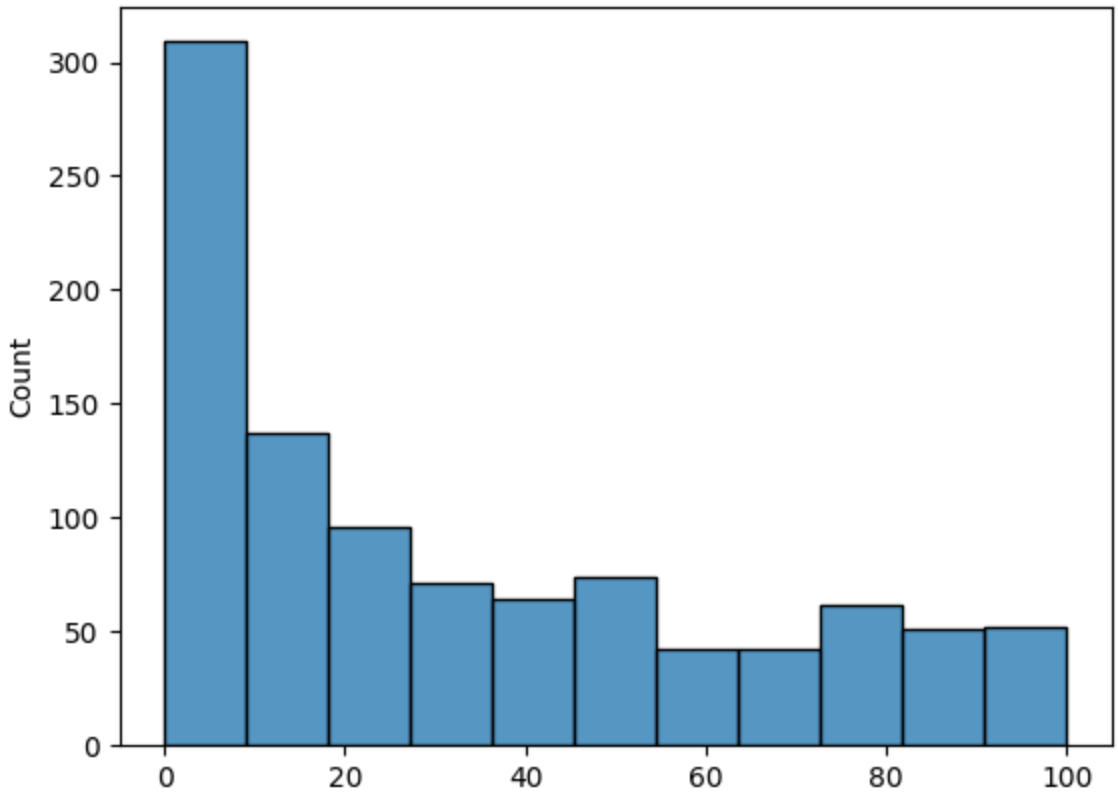
Out[3]:
```
array([30.12, 51.15, 36.33, 29.69, 17.95, 41.72, 19.15, 79.53, 92.86,
       14.7 ])
```

In [4]:
```
import seaborn as sns
```

In the plot below, you can see a histogram of the values in the series. For some reason, it decided to have exactly 11 bins (we allowed it to choose the number of bins.) Most values - about 300 of them - are between 0 and 9, and the next most likely bin is between 9 and 18. Since there are 1000 values, the total of the bars should be 1000.

In [5]:
```
sns.histplot(lesson_series)
```

Out[5]:
```
<Axes: ylabel='Count'>
```



There are 11 bins or bars, a number which by default is chosen by seaborn. We can reproduce this manually to (hopefully) see the same values as numbers. I'm not sure that this second histogram is guaranteed to be exactly the same, but it looks the same to me:

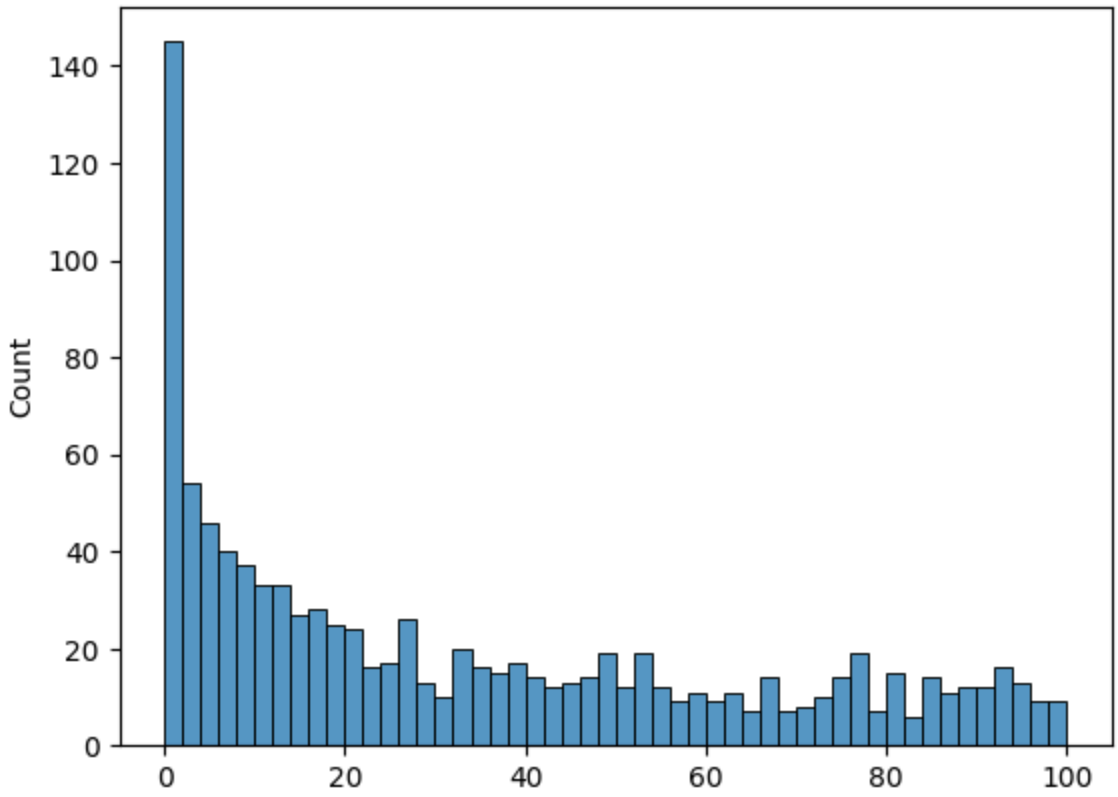
In [6]:
```
np.histogram(lesson_series, bins = 11)[0]
```

Out[6]:
```
array([309, 137,  96,  71,  64,  74,  42,  42,  62,  51,  52])
```

What happens if we override seaborn and choose the number of bins ourselves? We could choose a much larger number of bins:

In [7]:
```
sns.histplot(lesson_series, bins = 50)
```

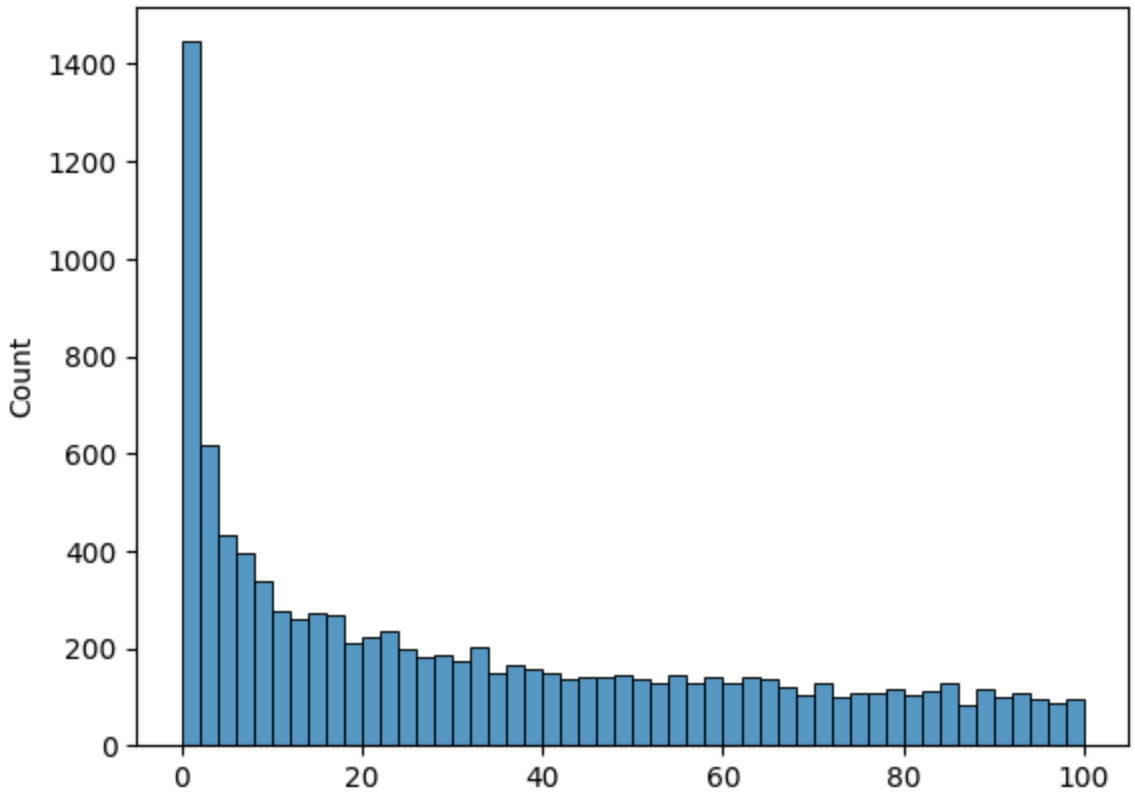
Out[7]:
```
<Axes: ylabel='Count'>
```



One disadvantage of this 50 bin picture is that the outliers are worse. That is, the graph wobbles up and down a bit more randomly. That's because there are fewer values in each bin, so there's more of a role for chance to take effect. If we had many more data points and/or fewer bins, we could get rid of this wobble.

In [8]:
```
lesson_series_2 = np.round(np.random.random(size = 10000)**2 * 100, 2)
sns.histplot(lesson_series_2, bins = 50)
```

Out[8]:
```
<Axes: ylabel='Count'>
```

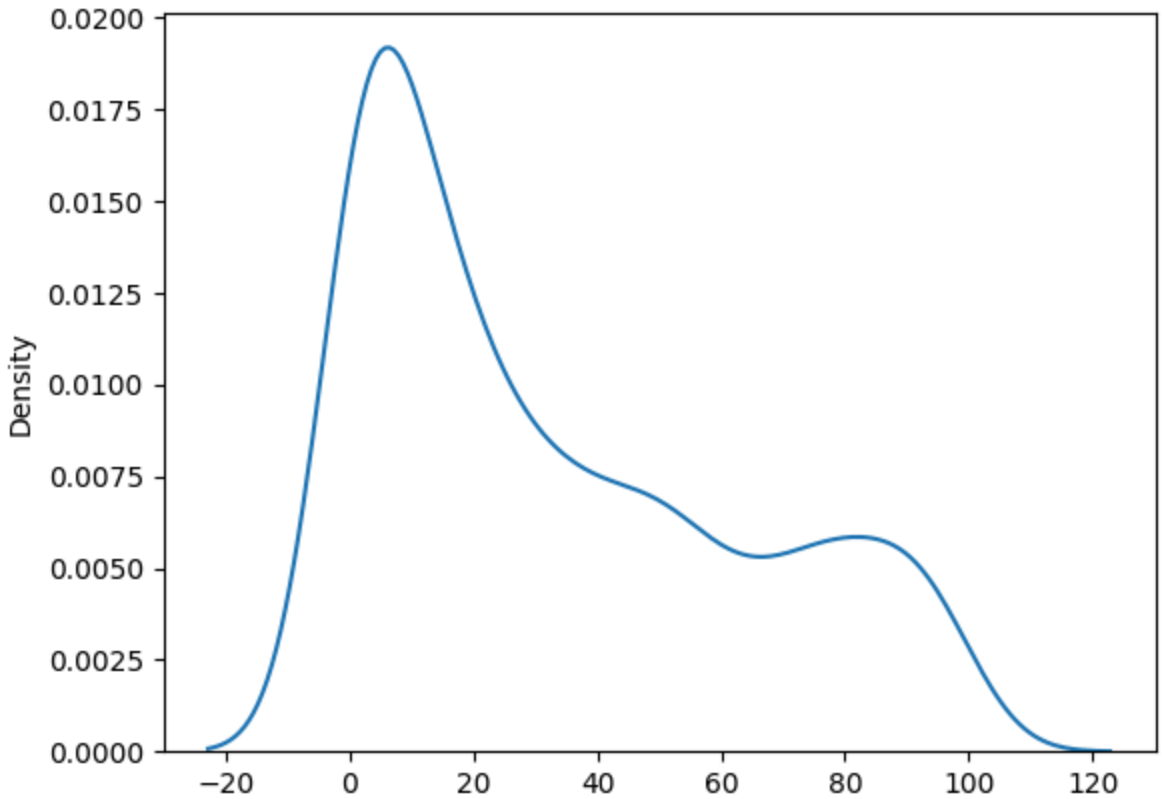


Here you can see that because the number of bins is the same as in the second graph above, but for more data, the histogram is a smoother graph. Why does more data make for a smoother graph? Something for you to think about. I said it's because a larger number of data points in each bin reduces the role of chance - but why is that?

Here is a KDE (Kernel Density Estimate) plot. It's just the same histogram, but drawn smoothly. The KDE plot doesn't have a "number of bins." It's always drawn the same way. In this case, because of the smoothness of the curve, it seems that x-values less then zero and above 100 are still plotted, even though there were no such values in the dataset. This seems like a drawback of the KDE plot, especially if the viewer is unprepared for this aspect of the plot.

In [9]:
```
sns.kdeplot(lesson_series)
```

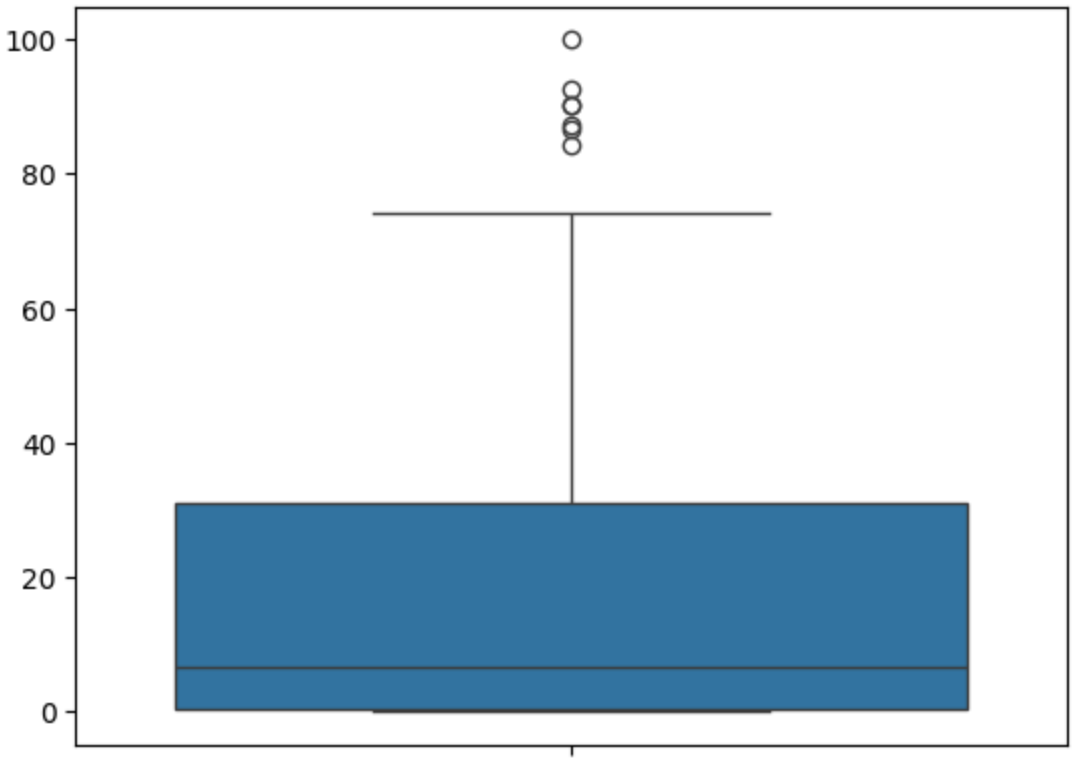
Out[9]:
```
<Axes: ylabel='Density'>
```



We could also draw a box plot. This time, to construct the data I used a fourth power rather than squaring, with only 100 data points, so that the points are even more concentrated toward the small numbers. It turns out that this will create a more interesting boxplot. The top and bottom edges of the box are the 75th and 25th percentile, respectively, and the top and bottom "whiskers" show a larger range which is a multiple of 1.5 times the the box height. (The bottom whisker cannot be see because it's pushed against the bottom of the graph.) The filled-in box shows that half of the values are between about 0 and 30 on the y-axis. Is that what you'd expect? The 25th and 75th percentile of the original uniform random variable are at 0.25 and 0.75. Taken to the fourth power and multiplied by 100 (remember, that's how we constructed our sample), that's 0.25**4 * 100 = 0.4 and 0.75**4 * 100 = 32. It's plausible that those are the height of the bottom and top of the box. We can see that a small number of samples are above the top whisker; they are shown as individual dots.

In [10]:
```
lesson_series_3 = np.round(np.random.random(size = 100)**4 * 100, 2)
sns.boxplot(lesson_series_3)
```

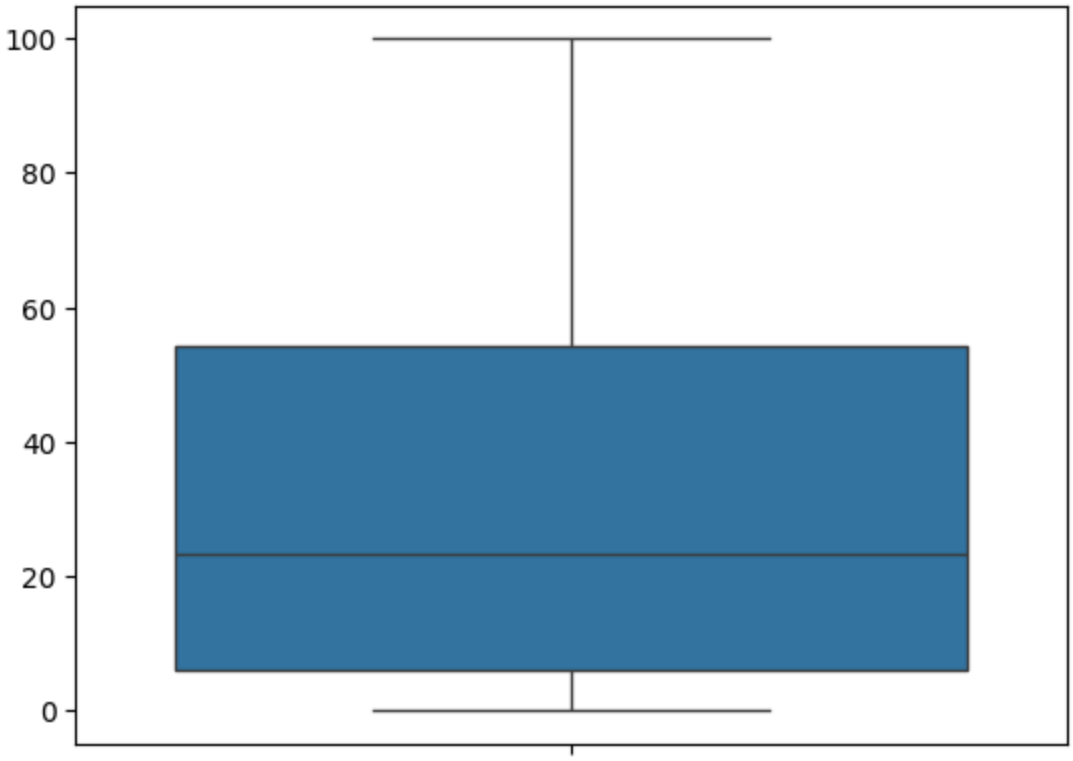
Out[10]: <Axes: >



If we go back to the original lesson_series with the squared values, there will be two whiskers, because it isn't so strongly weighted toward small values:

In [11]:
```
sns.boxplot(lesson_series)
```

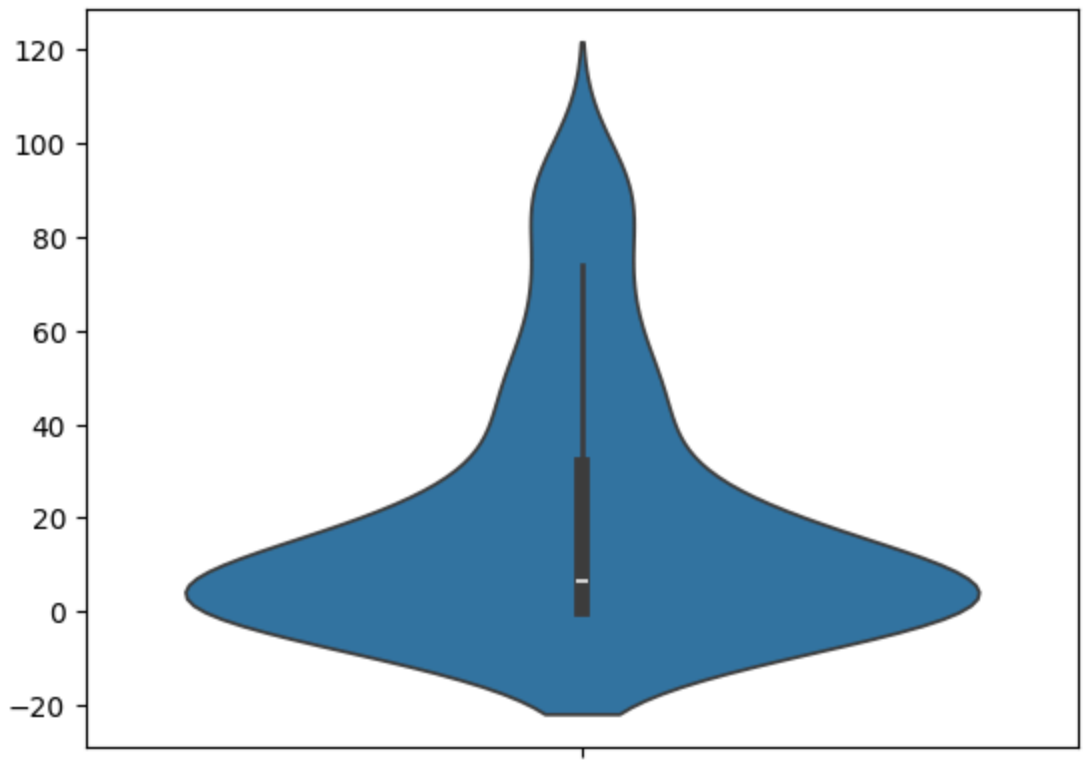
Out[11]: <Axes: >



Going back to the fourth power series, another histogram variant is the violin plot. This simply combines a kde plot (turned on its side and forming two side of the violin) with a boxplot:

In [12]:
```
sns.violinplot(lesson_series_3)
```

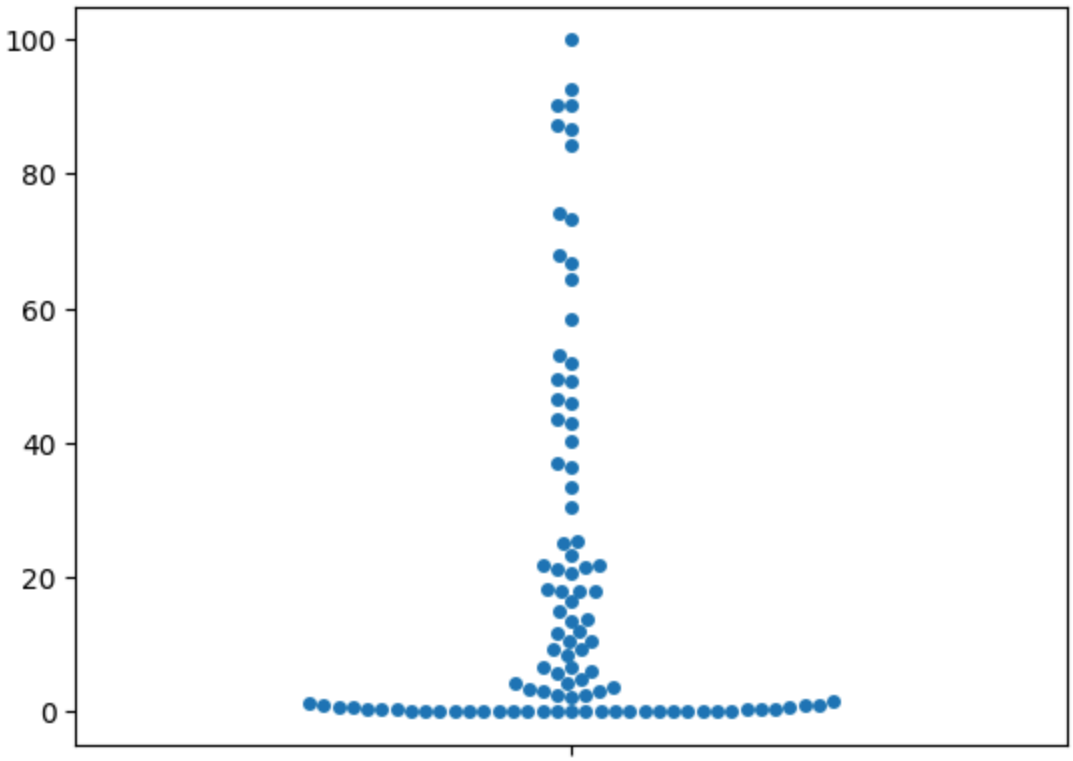
Out[12]: <Axes: >



Finally, a swarm plot shows the histogram (turned on its side and doubled, as with the violin plot) but showing each individual point.

In [13]:
```
sns.swarmplot(lesson_series_3)
```

Out[13]: <Axes: >



2. Weekly graph question

Below are a histogram and table representation of the same data. A species of bird is being analyzed, and each individual's body length in inches has been measured.

Please write a short explanation of the pros and cons of these two representations. Which would you choose? How would you modify the formatting, if at all, to make it more visually interesting, clear, or informative?

In [14]:
```
import numpy as np
import pandas as pd

np.random.seed(0)
num_data = 10000
data = np.random.normal(size = num_data) + 6
df = pd.DataFrame(data.T, columns = ["data"])
```

In [15]:
```
histnums = np.histogram(df["data"])
histcounts = histnums[0]
histmins = histnums[1][0:-1]
histmaxes = histnums[1][1:]
```

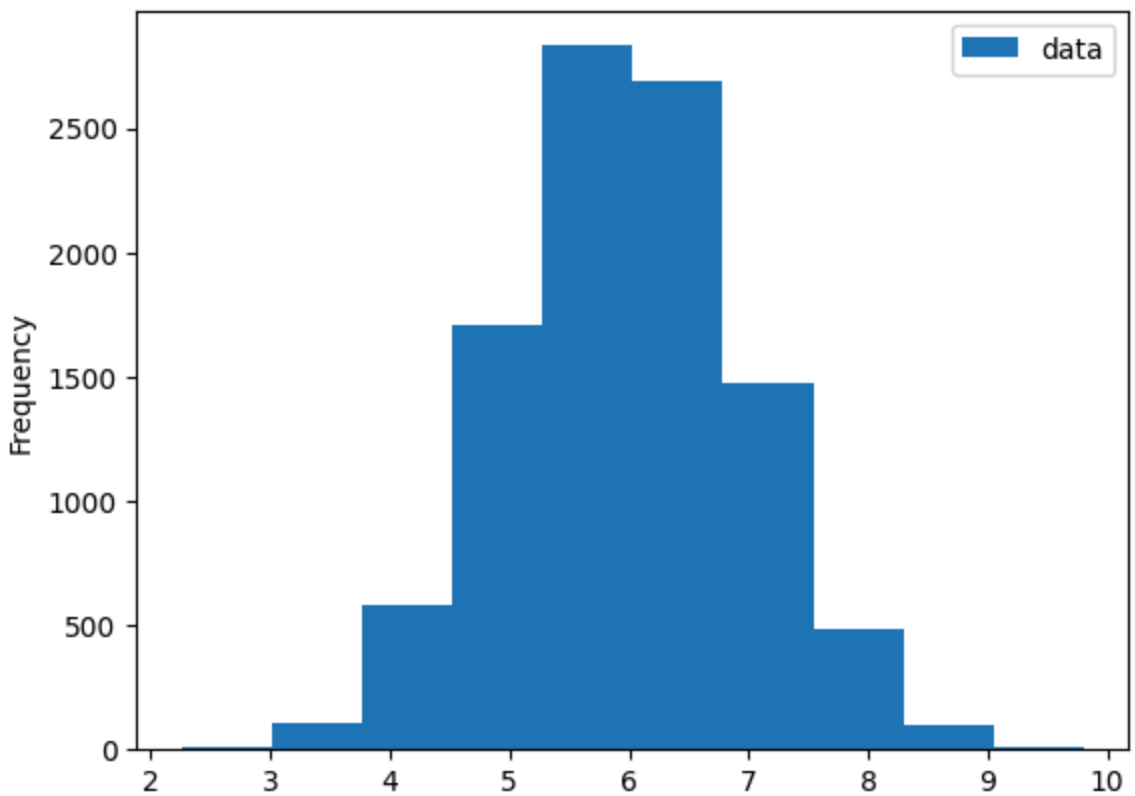
In [16]:
```
pd.DataFrame(np.array([histcounts, histmins, histmaxes]).T, columns = ["count", "mi
```

Out[16]:

	count	minval	maxval
0	10.0	2.259899	3.014075
1	110.0	3.014075	3.768252
2	579.0	3.768252	4.522428
3	1710.0	4.522428	5.276604
4	2833.0	5.276604	6.030780
5	2688.0	6.030780	6.784956
6	1479.0	6.784956	7.539132
7	487.0	7.539132	8.293308
8	97.0	8.293308	9.047484
9	7.0	9.047484	9.801660

In [17]:
```
df.plot.hist()
```

Out[17]: <Axes: ylabel='Frequency'>



The DataFrame and Plot Histogram each have benefits and disadvantages.

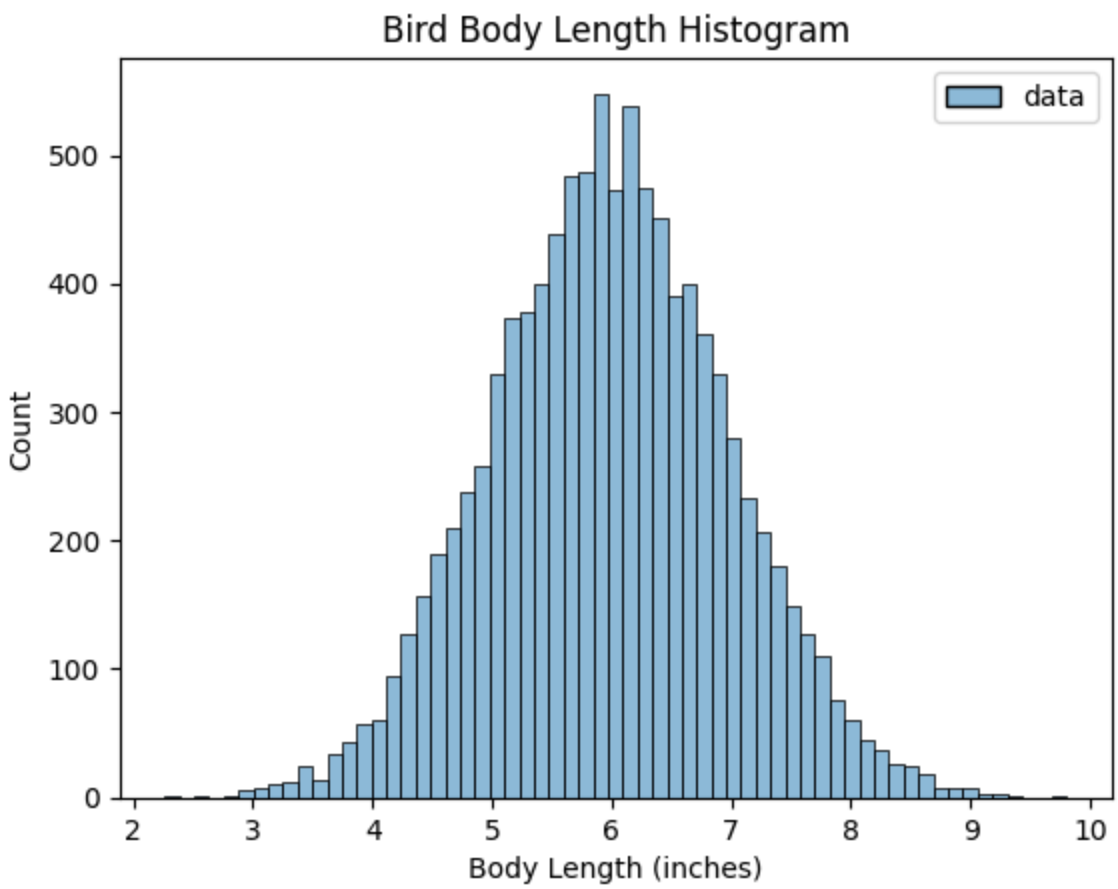
The DataFrame Histogram provides more granuarity into the data itself. You can clearly see what values each histogram bin contains, the number of bins, and the count of instances in each bin. However, the with the DataFrame it is harder to make inferences into how the

distribution is shaped. Your eyes need to dart back and forth between each row to make comparisons.

The Plot Histogram is much easier to read and visualize the distribution of the data. However, it does not contain the granuarity of what instances each bin contains to the decimal point.

I would generally utilize the histogram since it gives a more easily digestible representation of the data. A few improvements can be made to the plot, like adding a title, labeling the X axis, and making the legend more clear. Additionally, by increasing the number of bins we get a better idea of the distribution. I have made those changes below..

In [18]:
```
import matplotlib.pyplot as plt
sns.histplot(
    df
)
plt.title("Bird Body Length Histogram")
plt.xlabel("Body Length (inches)")
None
```



3. Homework - Amusement Park Rides

Now let's imagine we have some data about how many times different visitors to an amusement park used each ride, as well as how much money they spend at the amusement park. Each sample represents a single visit by a single visitor on a given date.

In [19]:
```
num_visits = 10000
np.random.seed(0)
```

In [20]:
```
df = pd.DataFrame(columns = ["VisitDate"])
```

In [21]:
```
start = datetime(2010, 1, 1)
end = datetime(2024, 1, 1)
numdays = (end - start).days
random_days = np.random.randint(0, numdays, size = num_visits)
s = start + pd.to_timedelta(random_days, unit='D')
s = s.sort_values()
df["VisitDate"] = s
```

In [22]:
```
df["IsAdult"] = np.random.choice([True, True, False], size = num_visits)
```

In [23]:
```
df["MartianRide"] = np.random.choice([0] * 8 + [1] * 3 + [2] * 3 + [3] * 1 + [10],
```

In [24]:
```
df["TeacupRide"] = np.random.choice([0] * 2 + [1] * 5 + [2] * 3 + [5] * 2, size = n
```

In [25]:
```
df["RiverRide"] = np.random.choice([0] * 8 + [1] * 3 + [2] * 2, size = num_visits)
```

In [26]:
```
df["MoneySpent"] = np.round(np.random.random(size = num_visits)**2 * 100, 2)
```

In [27]:
```
df.iloc[0:5]
```

Out[27]:

	VisitDate	IsAdult	MartianRide	TeacupRide	RiverRide	MoneySpent
0	2010-01-01	False	0	1	4	10.30
1	2010-01-01	True	3	0	0	38.77
2	2010-01-01	True	0	0	0	79.34
3	2010-01-01	True	0	0	1	87.84
4	2010-01-02	False	0	1	2	18.65

1. Find the mean, median, and mode for how many times visitors rode each ride. See Week 1 for Google advice on this.

In [28]:
```
def mode(x):
    return x.mode().iloc[0] if not x.mode().empty else None

ride_cols = ["MartianRide", "TeacupRide", "RiverRide"]

df[ride_cols].agg(['mean', 'median', mode])
```

Out[28]:

	MartianRide	TeacupRide	RiverRide
mean	0.9073	0.5862	1.2007
median	0.0000	0.0000	1.0000
mode	0.0000	0.0000	0.0000

2. Use groupby() to find the mean, median, and mode for how many times each ride was ridden on each given day.

Suggested Google search or ChatGPT prompt: I first tried: "How do I use groupby to find the mean over each day in my DataFrame?" but Google didn't help me.

Then I tried: "How do I find the mean over each date in a dataframe?" Sometimes, you have to try multiple searches.

This gives me a very helpful site, where someone is doing the same thing we are:

https://stackoverflow.com/questions/40788530/how-to-calculate-mean-of-some-rows-for-each-given-date-in-a-dataframe

In [29]:
```
df.groupby(by='VisitDate')[ride_cols].agg(['mean', 'median', mode])
```

Out[29]:

	MartianRide			TeacupRide			RiverRide		
	mean	**median**	**mode**	**mean**	**median**	**mode**	**mean**	**median**	**mode**
VisitDate									
2010-01-01	0.75	0.0	0	0.25	0.0	0	1.25	0.5	0
2010-01-02	0.00	0.0	0	1.00	1.0	1	2.00	2.0	2
2010-01-03	0.00	0.0	0	1.50	0.5	0	1.25	0.5	0
2010-01-04	0.00	0.0	0	0.00	0.0	0	1.00	1.0	1
2010-01-05	0.50	0.5	0	0.00	0.0	0	0.00	0.0	0
...	...	...	...	...	...	...	...	...	...
2023-12-27	0.50	0.5	0	0.50	0.5	0	1.00	1.0	0
2023-12-28	6.00	6.0	2	0.00	0.0	0	0.00	0.0	0
2023-12-29	0.00	0.0	0	1.00	1.0	1	3.00	3.0	3
2023-12-30	3.00	3.0	3	0.00	0.0	0	1.00	1.0	1
2023-12-31	0.00	0.0	0	0.00	0.0	0	1.00	1.0	1

4404 rows × 9 columns

3. Find the standard deviation and variance of the count for each ride.

Suggested Google search or ChatGPT prompt: "How do I find the standard deviation of a Series in pandas documentation?" and similar query for variance.

I included the word "documentation" because this task likely involves applying one single function for standard deviation and another for variance, so I expected that I could find a single documentation page for each that would cover my needs. I get these:

https://pandas.pydata.org/pandas-docs/dev/reference/api/pandas.Series.var.html

https://pandas.pydata.org/docs/reference/api/pandas.Series.std.html

In [30]:
```
df[ride_cols].agg(['std', 'var'])
```

Out[30]:

	MartianRide	TeacupRide	RiverRide
std	2.077339	1.232851	1.295757
var	4.315338	1.519922	1.678987

4. Find the 90th percentile count for each ride. That is, if the customers are ordered by their number of rides, and there are 100 customers, how many rides does the 90th person take? There is a function in pandas that easily does this calculation.

Suggested Google search or ChatGPT prompt: "How do I find a percentile value for a Series in pandas documentation?"

This time, I will let you find the page! Hint: it's not actually called the "percentile" function.

In [31]:
```
df[ride_cols].quantile(0.9)
```

Out[31]:
```
MartianRide    2.0
TeacupRide     2.0
RiverRide      3.0
Name: 0.9, dtype: float64
```

5. Plot a histogram of the ride count, using each day as data element as you did in #2. Use both the total ride count for each day as well as, separately, the mean ride count for each day.

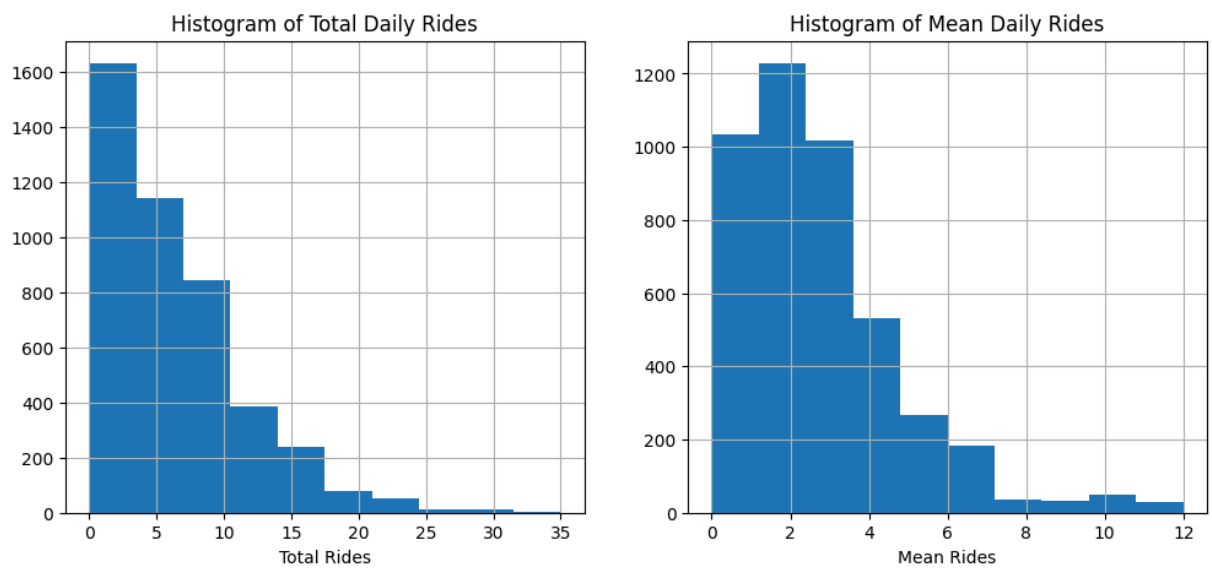
Suggested Google search or ChatGPT prompt: "How do I plot a histogram for a Series in pandas?"

I found this: https://pandas.pydata.org/docs/reference/api/pandas.Series.plot.hist.html

Note: the "by" keyword will not help you plot a count for each day; if you try to use it for this, it will try to draw a separate histogram for each day.

You've already learned how to find the mean ride count for each day. Can you use a similar idea to find the total ride count for each day?

In [32]:
```
df_copy = df.copy()
df_copy['TotalRideCount'] = df_copy['MartianRide'] + df_copy['TeacupRide'] + df_cop

total_ride_stats = df_copy.groupby('VisitDate')['TotalRideCount'].agg(['sum', 'mean

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

total_ride_stats['sum'].hist(ax=ax1)
ax1.set_title('Histogram of Total Daily Rides')
ax1.set_xlabel('Total Rides')

total_ride_stats['mean'].hist(ax=ax2)
ax2.set_title('Histogram of Mean Daily Rides')
ax2.set_xlabel('Mean Rides')

None
```



6. Make a bar chart showing the total number of Adult and Child participants.

Try to formulate your own Google search or ChatGPT prompt.

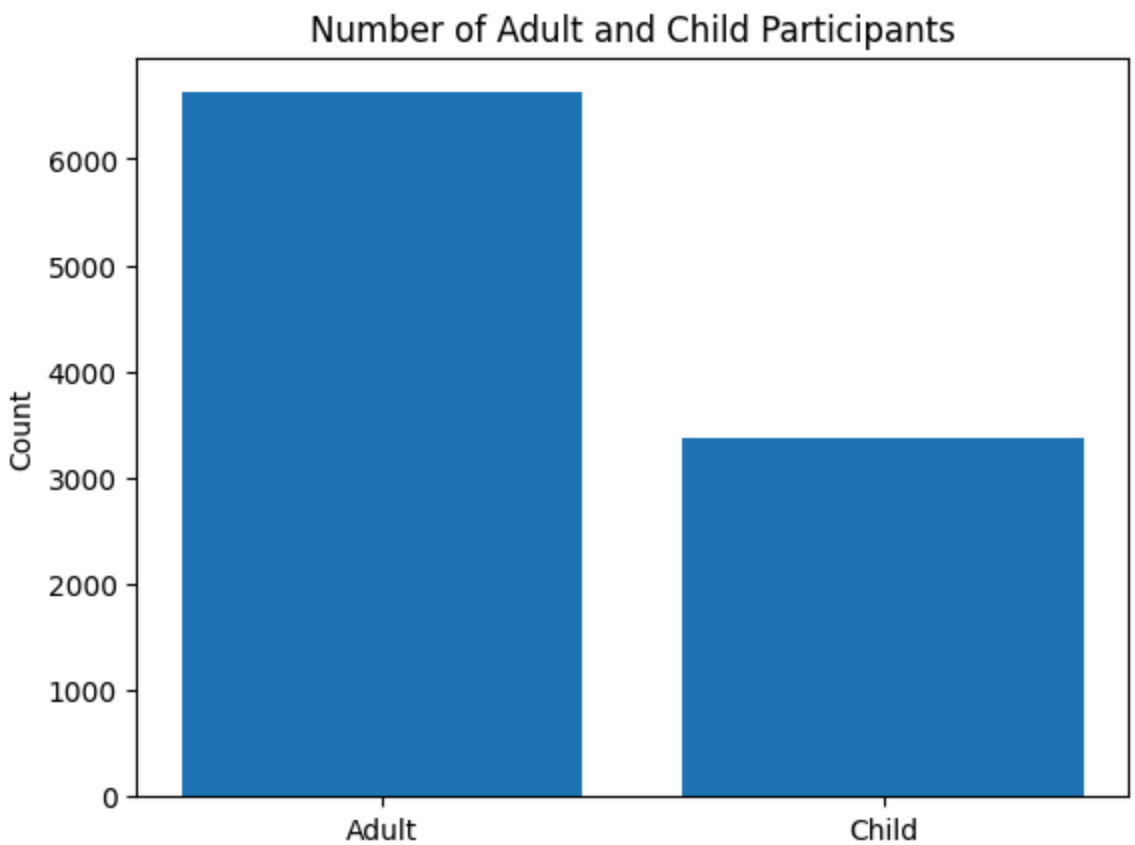
Here's one page I found:

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.bar.html

which shows some interesting examples for you to use. I recommend clicking on the first, basic example to see how to use the function.

You can also look up the pandas documentation, which might lead to an entirely different method.

In [33]:
```
num_adults = len(df[df['IsAdult'] == True])
num_children = len(df[df['IsAdult'] == False])
assert num_adults + num_children == len(df)
plt.bar(x=['Adult', 'Child'], height=[num_adults,num_children])
plt.title("Number of Adult and Child Participants")
plt.ylabel("Count")
None
```



7. Make a stacked bar chart showing how many Adult and Child visits took the River Ride, with the x-axis showing the number of rides and with two stacked bars (adult, child). That is, there could be a column for the number of Adults (and Children) who took 0 rides, 1 ride, 2 rides, and so on.

Try to formulate your own Google search or ChatGPT prompt.

For me, the pandas documentation was easier than the matplotlib documentation.

- If you use the pandas method, you might have to one-hot encode the IsAdult column. You might want names for the new columns other than True and False. Use df.join or pd.concat to attach the pd.get_dummies table you've created, and then groupby the RiverRide value.

- You will need to end up with a small table with two columns (IsAdult True and False) and an index (RiverRide) as well as a small number of rows (the values of RiverRide)
- That said, if you can come up with another approach, it's fine.

In [34]:
```
is_adult_one_hot = pd.get_dummies(df['IsAdult'], prefix='IsAdult', dtype=int)
is_adult_one_hot
```

Out[34]:

	IsAdult_False	IsAdult_True
0	1	0
1	0	1
2	0	1
3	0	1
4	1	0
...	...	...
9995	0	1
9996	0	1
9997	1	0
9998	0	1
9999	0	1

10000 rows × 2 columns

In [35]:
```
p7_df = df.copy().join(is_adult_one_hot).drop(columns=['IsAdult', 'MartianRide', 'T
p7_df
```

Out[35]:

	RiverRide	IsAdult_False	IsAdult_True
0	4	1	0
1	0	0	1
2	0	0	1
3	1	0	1
4	2	1	0
...	...	...	...
9995	0	0	1
9996	0	0	1
9997	3	1	0
9998	1	0	1
9999	1	0	1

10000 rows × 3 columns

In [36]:
```
adult_rides = p7_df[p7_df['IsAdult_True'] == 1]
adult_rides
```

Out[36]:

	RiverRide	IsAdult_False	IsAdult_True
1	0	0	1
2	0	0	1
3	1	0	1
6	0	0	1
7	0	0	1
...	...	...	...
9993	0	0	1
9995	0	0	1
9996	0	0	1
9998	1	0	1
9999	1	0	1

6625 rows × 3 columns

In [37]:
```
child_rides = p7_df[p7_df['IsAdult_False'] == 1]
child_rides
```

Out[37]:

	RiverRide	IsAdult_False	IsAdult_True
0	4	1	0
4	2	1	0
5	4	1	0
8	1	1	0
12	3	1	0
...	...	...	...
9971	3	1	0
9975	1	1	0
9981	1	1	0
9994	2	1	0
9997	3	1	0

3375 rows × 3 columns

In [38]:
```
child_rides_by_count = child_rides.groupby(by='RiverRide').sum()
child_rides_by_count
```

Out[38]:

RiverRide	IsAdult_False	IsAdult_True
1	841	0
2	844	0
3	798	0
4	892	0

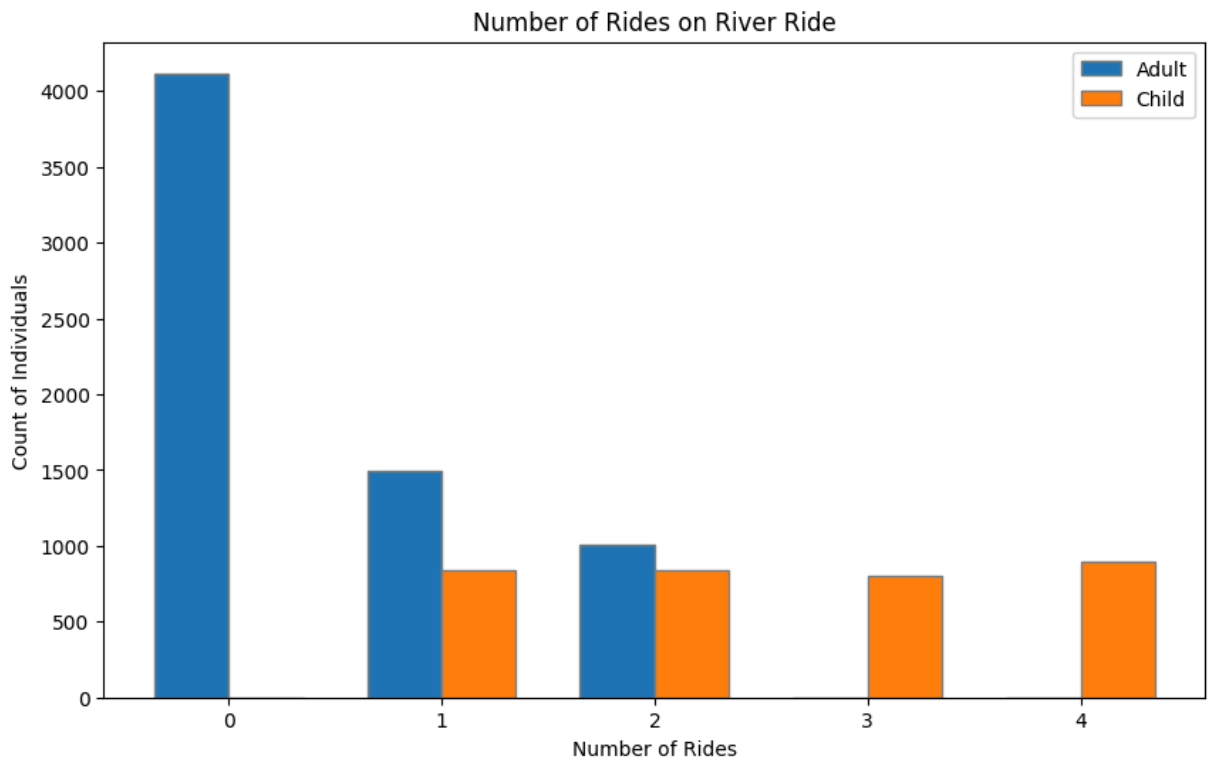
In [39]:
```
adult_rides_by_count = adult_rides.groupby(by='RiverRide').sum()
adult_rides_by_count
```

Out[39]:

RiverRide	IsAdult_False	IsAdult_True
0	0	4118
1	0	1498
2	0	1009

In [40]:
```
labels = ["0", "1", "2", "3", "4"]

# Adult Rides have values for ride count [0,1,2]. Child have values for [1,2,3,4].
```

```
adult_ride_counts = np.append(adult_rides_by_count['IsAdult_True'], [0,0])
child_ride_counts = np.append([0], child_rides_by_count['IsAdult_False'])

bar_width = 0.35
# Set position of bars on x-axis
r1 = np.arange(len(labels))
r2 = [x + bar_width for x in r1]

plt.figure(figsize=(10, 6))
plt.bar(r1, adult_ride_counts, width=bar_width, edgecolor='grey', label='Adult')
plt.bar(r2, child_ride_counts, width=bar_width, edgecolor='grey', label='Child')
plt.title("Number of Rides on River Ride")
plt.xlabel("Number of Rides")
plt.ylabel("Count of Individuals")
plt.xticks([r + bar_width/2 for r in r1], labels) # Center x-axis labels
plt.legend()
None
```



8. Other tasks

Use seaborn (import seaborn as sns) to create a kernel density estimation (kde) plot. (You can choose any aspect of the amusement park dataset to plot.) Here is a tutorial you can read about seaborn:

https://seaborn.pydata.org/tutorial/introduction.html

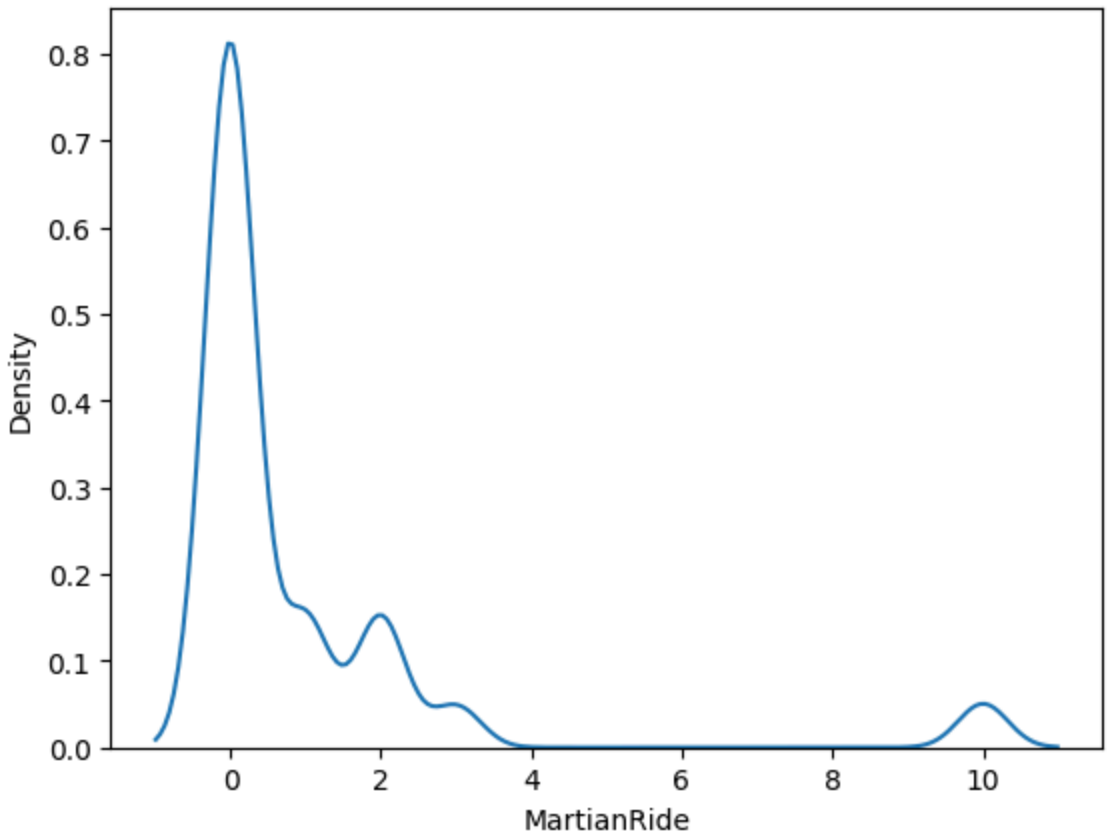
Seaborn often makes nicer looking graphs than pandas or matplotlib.

Now use seaborn to make a swarm plot, a violin plot, and a box plot.

Which plots are the best for showing this data?

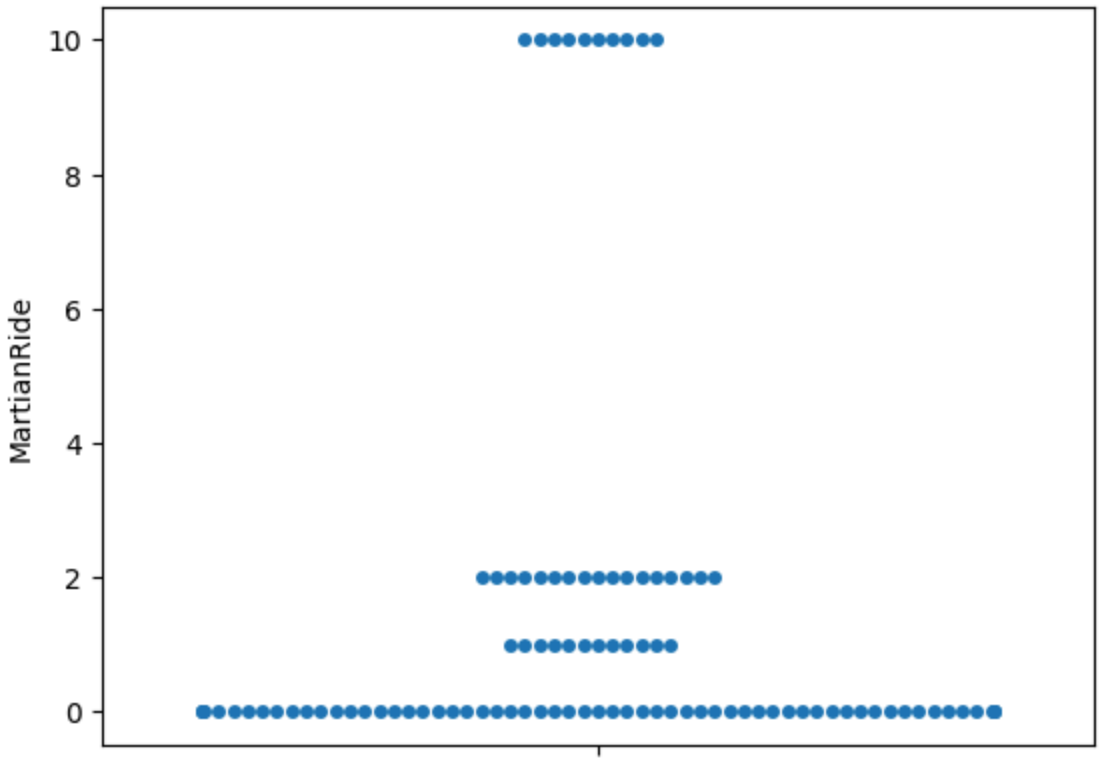
In [41]:
```
sns.kdeplot(df['MartianRide'])
```

Out[41]:
```
<Axes: xlabel='MartianRide', ylabel='Density'>
```

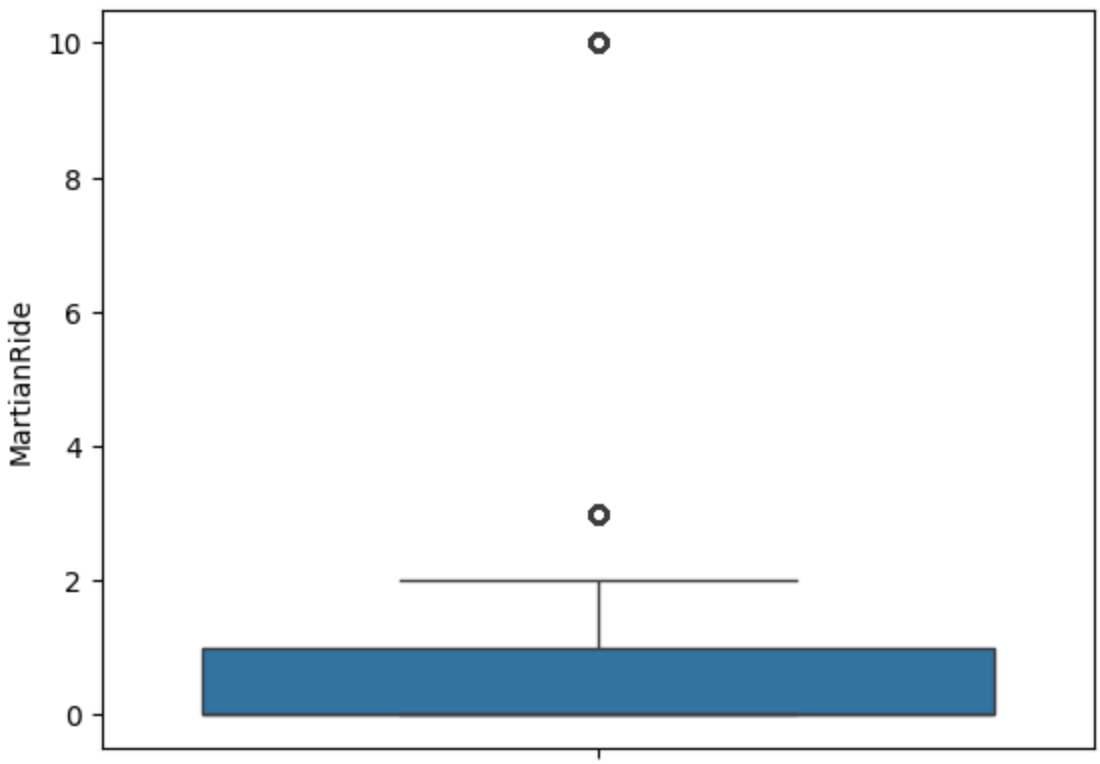


In [42]:
```
sns.swarmplot(df.sample(100)['MartianRide'])
None
```

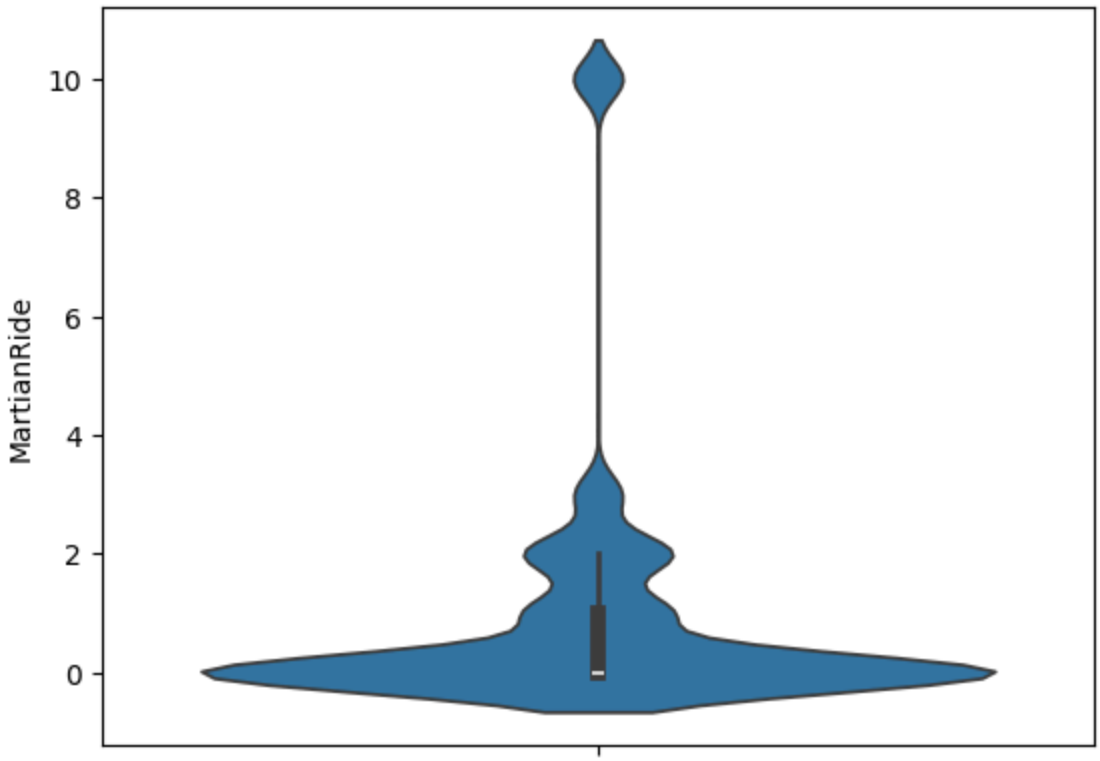
```
c:\Users\caleb\anaconda3\envs\iris-fisher\lib\site-packages\seaborn\categorical.py:3
399: UserWarning: 6.0% of the points cannot be placed; you may want to decrease the
size of the markers or use stripplot.
  warnings.warn(msg, UserWarning)
```



In [43]:
```
sns.boxplot(df['MartianRide'])
None
```



In [44]:
```
sns.violinplot(df['MartianRide'])
None
```



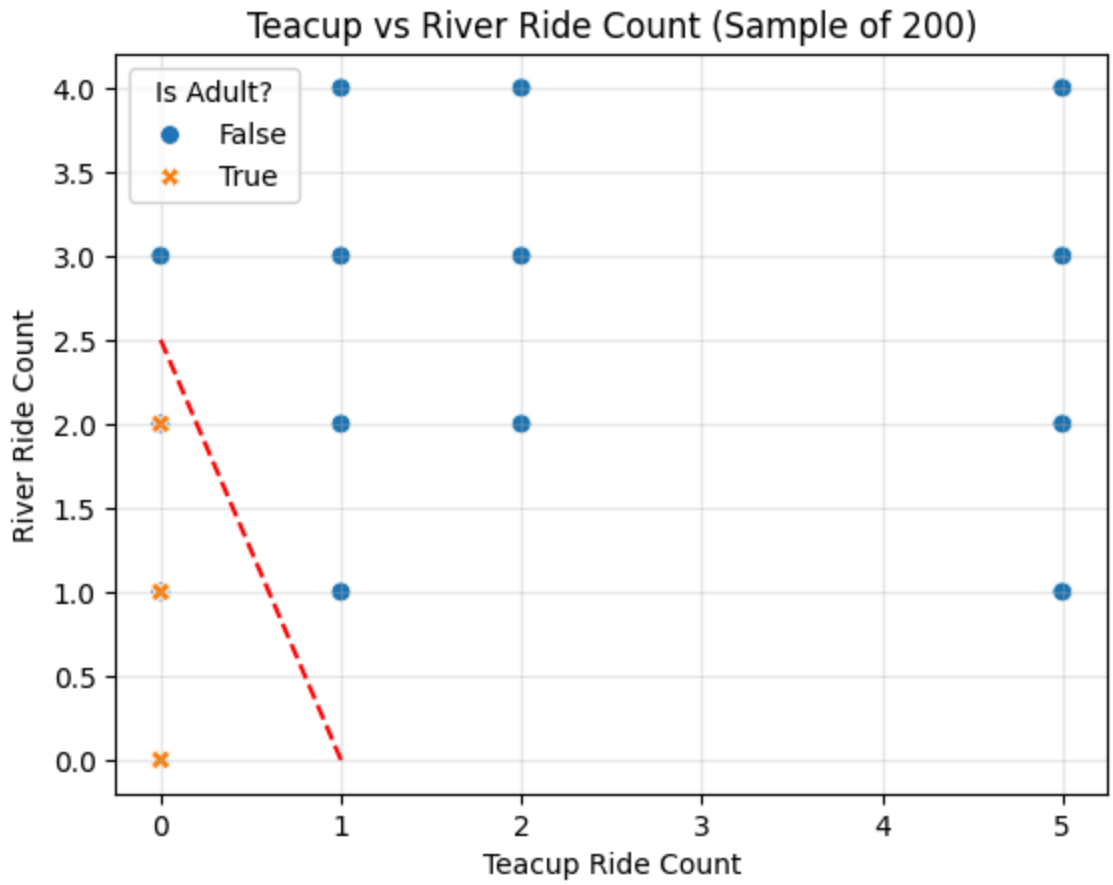
4. Storytelling With Data graph

Try to make a scatterplot using amusement park dataset that is similar to the one on page 45, where the two axes are the Teacup Ride count and the River Ride count. If you want to see individual dots and not a dense swarm of dots, you'll have to cut the dataset down to a small number of points (say, 100 points or so). You can remove the other points, for instance, and focus on the first 100 points.

Here are some things you could do (you don't have to do all of them):

- Draw a dashed line that roughly separates the adult from child points.
- Draw the points in the adult vs. child region of the graph in different colors.
- Choose the right size and number of dots to make the graph look good.
- Write a word that appears on the dashed line (like AVG in the plot on page 45).

If there are any other graphs in the Storytelling With Data chapter that look interesting, and you want more practice, you can try to reproduce them too.

In [82]:
```
sample = df.sample(200)

sns.scatterplot(data=sample, x='TeacupRide', y='RiverRide', hue='IsAdult', style='I
plt.plot([0, 1], [2.5, 0], color='red', linestyle='--')
plt.title('Teacup vs River Ride Count (Sample of 200)')
plt.xlabel('Teacup Ride Count')
```

```
plt.ylabel('River Ride Count')
plt.legend(title='Is Adult?')
plt.grid(True, alpha=0.3)
```